

Epic 1 Summary And Final Test Plan

Date: 2026-07-03

Status: Epic 1 closeout package prepared; E1.7 live Railway deployment remains gated

Scope: E1.1-E1.7 knowledge base, PRD workflow, dashboard/Jarvis, final testmeeting rerun, critique

Executive Summary

Epic 1 built the internal ArchFlow proof engine: approved product material can become source inventory, context digest, PRD, task/backlog package, review report, KB update, and dashboard/Jarvis review packet without storing raw private material in public files.

The strongest result is method reliability, not market demand. Epic 1 proves that ArchFlow can operate a controlled PRD/KB workflow on itself. It does not prove that buyers will pay, that the cloud runtime is complete, or that provider-backed Jarvis is safe to run by default.

Epic 1 Scorecard

Dimension	Score	Visual	Evidence
Source boundary discipline	5/5	#####	E1.4 principle, public safety scan, testmeeting sanitizer
PRD artifact generation	5/5	#####	E1.2 and E1.2.8 PRD/PDF packages
KB governance	4/5	####.	WikiLLM run notes, E1.3 readback, E1.4 accepted principle
Dashboard/Jarvis review surface	4/5	####.	Vercel provider-disabled API and local dashboard packets
Cloud runtime independence	2/5	##...	Railway scaffold exists; deployment and health proof pending
Provider/model readiness	2/5	##...	OpenRouter comparison exists as review-gated artifact; default provider disabled
Market/customer proof	1/5	#....	No outreach, payment, or firm paid intent yet

Epic 1 Task State

Task	Status after this run	Evidence	Remaining gap
E1.1 Separate KB project	Done/reviewed	Project plan and public repo structure	None for current scope
E1.2 Dialogue to PRD cycle	Review, strong proof	June 26 proof and July 3 testmeeting rerun	Owner Done-state decision
E1.2.8 testmeeting PRD/PDF	Rerun locally on 2026-07-03	project/runs/E1.2/2026-07-02-testmeeting-local/	OpenRouter not rerun in this pass
E1.3 KB write/readback	Review	E1.3 readback run notes	Owner acceptance and future memory hygiene
E1.4 KB update principle	Accepted	project/reports/2026-07-03-kb-update-principle.md	Apply consistently
E1.5 Case-study/content process	In progress/review	Reporting gate, content templates, dashboard reports	Final content pack after E2 evidence
E1.6 Personal KB	Separate task	Project plan keeps it separate	Not part of Railway runtime

Task	Status after this run	Evidence	Remaining gap
E1.7 Hosted dashboard/Jarvis/API	Plan/scaffold, not complete	Vercel proof and Railway scaffold	Railway deploy, health, CORS/auth, dashboard routing

Final Test Run: testmeeting

The final local test reran `project/scripts/e1_2_8_testmeeting_run.py` against the approved private fixture and methodology document. The public outputs remain sanitized derived artifacts only.

Test item	Result
Fixture present	Yes
Local runner completed	Yes
Stream events	6 events
OpenRouter status	not_run in this final baseline
Raw private text stored in public output	No by design; public output is sanitized
Main product focus	Release kickoff and discovery-to-delivery handoff modernization
Buyer hypothesis	Director/VP Product in B2B SaaS product team

What The Setup Provides

For ArchFlow:

- a repeatable internal proof workflow;
- source-bound PRD generation;
- public-safe run reporting;
- a review-packet dashboard/Jarvis surface;
- WikiLLM run memory and provenance discipline;
- a clear bridge from Epic 1 proof to Epic 2 market research.

For clients:

- a service concept for turning messy product-team material into reviewed execution artifacts;
- artifacts that can include PRD, backlog, acceptance criteria, questions, risks, decisions, and KB handoff;
- a visible review/safety method rather than ungrounded AI output.

How To Use The Epic 1 Result

- Use E1.2.8 as the demo fixture for the PRD Pack method.
- Use E1.4 as the rulebook for all memory and evidence promotion.
- Use the dashboard/Jarvis review-packet surface as the operator cockpit.
- Use E2.0A as the gate before real public account research.
- Do not claim customer demand until E6/E7 produce paid or firm-intent proof.

Current Configuration Overview

Component	Current configuration	Safe default
Codex	local operator/editor/reviewer	Human-approved side effects
LangGraph	path/state/review architecture	Controller, not browser runtime claim
CrewAI	role structure and proof lane	Not default runtime
LlamaIndex	bounded retrieval layer	lexical/hybrid fallback, not memory
WikiLLM	durable public-safe memory	accepted facts only
Dashboard	static/operator UI	browser-local packets until reviewed
Jarvis API	Vercel provider-disabled review contract; Railway scaffold	provider calls zero
Railway	planned E1.7 backend	not deployed until approved
Providers	disabled by default	MODEL_PROVIDER=none

Critique

Epic 1 is strong as an internal operating proof and service-delivery method. It is weak as a product proof. The cloud runtime is not yet independent enough to call it a product, and the market proof is not started. The project should resist adding more dashboard features until E2 proves which buyer pain matters and E1.7 proves the smallest hosted backend safely.

Finish Criteria For Epic 1

Epic 1 can be considered complete for internal proof when:

- E1.4 accepted principle is applied.
- E1.2/E1.2.8 testmeeting artifacts pass validation.
- Epic 1 summary PDF is attached to Notion/GitHub.
- E1.7 Railway setup/test plan exists.
- Any remaining runtime deployment gap is explicitly moved to E1.7 follow-up, not hidden.

Epic 1 cannot be considered complete for full cloud runtime until Railway /health, CORS/auth, dashboard routing, and provider-disabled endpoint tests pass from the hosted service.

Notion Top-Of-Page Summary

Insert this at the top of the Epic 1 Notion page after push:

Epic 1 status, 2026-07-03: internal PRD/KB proof is complete for method scope. E1.4 is accepted. The testmeeting PRD/PDF package